# گزارش فنی مسیر تحلیل و پیاده‌سازی پروژه

# Stack Overflow Churn Prediction

---

## فصل ۲ — مهندسی داده و ساخت Event Log

---

### ۲.۱ مقدمه

#### ۲.۱.۱ هدف این فصل

در این فصل توضیح می‌دهیم که چگونه داده‌های خام Stack Overflow را به یک **Event Log** یکپارچه تبدیل کردیم که برای مدل‌سازی Churn قابل استفاده باشد.

**Event Log چیست؟** یک جدول ساده که همه‌ی فعالیت‌های همه‌ی کاربران را به‌ترتیب زمانی لیست می‌کند. به زبان ساده‌تر، مثل یک «دفتر روزانه» است که هر کاری که هر کاربر انجام داده را با تاریخ و ساعت ثبت کرده.

#### ۲.۱.۲ چرا Event Log لازم است؟

داده‌های خام Stack Overflow در چندین جدول جداگانه هستند:

- جدول `Posts` : سؤالات و پاسخ‌ها
- جدول `Comments` : نظرات
- جدول `PostHistory` : تاریخچه‌ی ویرایش‌ها و تغییرات
- جدول `Users` : اطلاعات کاربران
- جدول `Votes` : رأی‌ها ❌ (استفاده نشد — توضیح در ادامه)

برای تحلیل رفتار کاربر، نیاز داریم همه‌ی این فعالیت‌ها را در یک جدول واحد ترکیب کنیم. چرا؟ چون می‌خواهیم **توالی** فعالیت‌های کاربر را ببینیم، نه فقط تعدادشان را.

> 💡 **مثال مفهومی:**
>
> فرض کنید می‌خواهید بدانید کاربر **user_id=12345** در ماه گذشته چه کرده. بدون Event Log باید به ۴ جدول مختلف نگاه کنید و خودتان اطلاعات را ترکیب کنید. با Event Log، یک query ساده همه را نشان می‌دهد:

```
> ۲۰۲۳-۰۶-۰۱ 08:30 | Create_Answer    | post_id=98765
> ۲۰۲۳-۰۶-۰۱ 09:15 | Create_Comment   | post_id=98760
> ۲۰۲۳-۰۶-۰۳ 14:20 | Edit_Body        | post_id=98765
> ۲۰۲۳-۰۶-۰۵ 11:00 | Create_Question  | post_id=99001
```

> این نمای زمانی فوراً نشان می‌دهد که کاربر در ۵ روز اول ماه فعال بوده، اما بعد از آن چه شده؟

### ۲.۱.۳ چرا Vote را حذف کردیم؟

**مشکل اساسی:** در جدول `Votes` فقط **رأی‌گیرنده** (صاحب پست) مشخص است، نه **رأی‌دهنده**.

جدول Votes در Stack Overflow:

| CreationDate | VoteTypeId | PostId | VoteId |
|---|---|---|---|
| 2023-06-01 | (UpVote) 2 | 98765 | 123 |

این جدول می‌گوید پست ۹۸۷۶۵ یک UpVote گرفته، اما **نمی‌گوید چه کسی** این رأی را داده است! این یک محدودیت در ساختار داده‌ی عمومی Stack Overflow است.

**نتیجه:** نمی‌توانیم Vote را به‌عنوان «فعالیت کاربر» ثبت کنیم، چون نمی‌دانیم کدام کاربر رأی داده. بنابراین Vote از Event Log حذف شد.

---

## ۲.۲ منابع داده

### ۲.۲.۱ جداول MySQL اصلی

داده‌ها از Stack Overflow Data Dump استخراج و در MySQL وارد شده‌اند. فایل `import_users.py` نمونه‌ای از نحوه‌ی وارد کردن داده از XML به MySQL است:

| جدول | تعداد رکورد (تقریبی) | توضیح |
|---|---|---|
| `users` | ~۲۴ میلیون | همه‌ی کاربران ثبت‌نام‌شده |
| `posts` | ~۵۸ میلیون | سؤالات (PostTypeId=1) و پاسخ‌ها (PostTypeId=2) |
| `comments` | ~۸۳ میلیون | نظرات زیر پست‌ها |
| `posthistory` | ~۳۰۵ میلیون | همه‌ی تغییرات روی پست‌ها |

### ۲.۲.۲ فیلتر کاربران: ۹۲,۰۰۰ کاربر هدف

از ۲۴ میلیون کاربر، فقط **~۹۲,۰۰۰ کاربر** را که در پژوهش قبل به 20 پرسونا خوشه بندی شده بودند ، انتخاب کردیم تا نتایج باقی تحقیق مبتنی بر این پرسونا ها باشد .

### ۲.۲.۳ تبدیل MySQL به Parquet

برای سرعت بیشتر در پردازش، همه‌ی جداول به فرمت **Parquet** تبدیل شدند:

| جدول MySQL | اسکریپت تبدیل | فایل خروجی |
|---|---|---|
| users | `users_to_parquet.py` | `/filtered_users_parquet` |
| posts | `filtered_posts_to_parquet.py` | `/filtered_posts_parquet` |

| فایل خروجی | اسکریپت تبدیل | جدول MySQL |
|---|---|---|
| `/comments_parquet` | `comments_to_parquet.py` | comments |
| `/posthistory_parquet` | `posthistory_to_parquet.py` | posthistory |

**چرا Parquet؟**

1. **فشرده‌سازی:** ~۱۰ برابر کوچک‌تر از CSV
2. **سرعت:** خواندن ستون‌های خاص بسیار سریع‌تر
3. **سازگاری:** با Spark، DuckDB، Pandas، Polars کار می‌کند

---

# ۲.۳ انواع فعالیت (Action Types)

## ۲.۳.۱ فایل ActionTypes.csv

ما یک فایل مرجع داریم ( `ActionTypes.csv` ) که همه‌ی انواع فعالیت را تعریف می‌کند:

### دسته ۱: ایجاد محتوا (مهم‌ترین)

| ActionTypeId | نام | توضیح ساده |
|---|---|---|
| 1 | `Create_Question` | کاربر سؤال جدید می‌پرسد |
| 2 | `Create_Answer` | کاربر پاسخ می‌نویسد |
| 70 | `Create_Comment` | کاربر نظر می‌دهد |

این سه فعالیت **اصلی‌ترین** شاخص‌های Engagement هستند.

### دسته ۲: ویرایش

| ActionTypeId | نام | توضیح ساده |
|---|---|---|
| 13 | `Edit_Title` | ویرایش عنوان سؤال |
| 14 | `Edit_Body` | ویرایش متن پست |
| 15 | `Edit_Tags` | ویرایش تگ‌های سؤال |
| 16 | `Rollback_Title` | بازگشت عنوان به نسخه‌ی قبلی |
| 17 | `Rollback_Body` | بازگشت متن به نسخه‌ی قبلی |
| 18 | `Rollback_Tags` | بازگشت تگ‌ها به نسخه‌ی قبلی |

### دسته ۳: مدیریت پست (Moderation)

| ActionTypeId | نام | توضیح ساده |
|---|---|---|
| 30 | `Close_Post` | بستن پست (نیاز به ۳K+ Rep) |
| 31 | `Reopen_Post` | بازگشایی پست |
| 32 | `Delete_Post` | حذف پست |
| 33 | `Undelete_Post` | بازیابی پست حذف‌شده |
| 34 | `Lock_Post` | قفل کردن پست (فقط Moderator) |
| 35 | `Unlock_Post` | باز کردن قفل |
| 43 | `Protect_Question` | محافظت از سؤال |
| 44 | `Unprotect_Question` | برداشتن محافظت |

**دسته ۴: سایر**

| ActionTypeId | نام | توضیح ساده |
|---|---|---|
| 40 | `Community_Owned` | تبدیل به مالکیت جامعه |
| 50 | `Add_Post_Notice` | اضافه کردن اعلان به پست |
| 51 | `Remove_Post_Notice` | حذف اعلان |
| 59 | `Wizard_Create` | ایجاد سؤال با Ask Wizard (ابزار جدید SO) |

## ۲.۳.۲ چرا Initial_Title/Body/Tags را حذف کردیم؟

در کد `create_eventlog_v2.py` می‌بینید که PostHistoryTypeIdهای ۱، ۲، ۳ کامنت شده‌اند:

```
python
PH_ACTION_MAP = {
    1: 10,   # Initial Title  ← حذف شده
    2: 11,   # Initial Body   ← حذف شده
    3: 12,   # Initial Tags   ← حذف شده
    4: 13,   # Edit Title
    5: 14,   # Edit Body
    ...
}
```

**دلیل:** وقتی کاربر یک سؤال می‌سازد ( `Create_Question` )، سیستم SO همزمان سه رکورد دیگر هم در PostHistory ثبت می‌کند: Initial_Body، Initial_Title و Initial_Tags. اگر همه را نگه می‌داشتیم، **یک فعالیت ۴ بار شمرده می‌شد!**

بنابراین فقط `Create_Question` را نگه داشتیم و Initialها را حذف کردیم.

## ۲.۴ ساخت Event Log (فایل create_eventlog_v2.py)

### ۲.۴.۱ ساختار کلی

این فایل سه منبع داده را ترکیب می‌کند:

```
Event Log = Posts (Question/Answer) + PostHistory (Edit/Delete/...) +
                                                              Comments
```

### ۲.۴.۲ مرحله ۱: بارگذاری جداول پایه

```
                                                              python
                                                    import polars as pl

                                                  کاربران فیلترشده با پرسونا #
                                                              users = (
                                        pl.scan_parquet("filtered_users_parquet/*.parquet")
                                                              .select([
                                                  pl.col("Id").alias("user_id"),
                                  pl.col("ClusterNmf20").alias("persona"),  # پرسونای NMF
                                     pl.col("CreationDate").alias("AccountCreationDate"),
                                                       pl.col("LastAccessDate"),
                                                                    ])
                                                                     )

                                                    پست‌های این کاربران #
                                posts = pl.scan_parquet("filtered_posts_parquet/*.parquet")

                                                  تاریخچه‌ی پست‌ها (ویرایش‌ها، حذف‌ها، ...) #
                                      ph = pl.scan_parquet("posthistory_parquet/*.parquet")

                                                                  نظرات #
                                   comments = pl.scan_parquet("comments_parquet/*.parquet")
```

**نکته‌ی فنی:** از `scan_parquet` (حالت Lazy) استفاده می‌کنیم. این یعنی Polars فوراً همه‌ی داده را به حافظه نمی‌خواند، بلکه فقط «برنامه‌ی اجرا» را می‌سازد و موقع `collect()` اجرا می‌کند. این برای داده‌های بزرگ ضروری است.

### ۲.۴.۳ مرحله ۲: استخراج رویدادهای Create Post

```python
CREATE_QUESTION_ACTION = 1
CREATE_ANSWER_ACTION = 2

events_create_post = (
    posts
    .join(users, left_on="owneruserid", right_on="user_id", how="inner")
    .with_columns([
        pl.col("owneruserid").alias("user_id"),
        pl.col("creationdate").alias("timestamp"),
        pl.col("id").alias("post_id"),
        # تبدیل PostTypeId به ActionTypeId
        pl.when(pl.col("posttypeid") == 1)
        .then(pl.lit(CREATE_QUESTION_ACTION))  # 1 → سؤال
        .when(pl.col("posttypeid") == 2)
        .then(pl.lit(CREATE_ANSWER_ACTION))    # 2 → پاسخ
        .otherwise(pl.lit(None))
        .alias("action_type_id"),
    ])
)
```

**توضیح ساده:**

- اگر `posttypeid = 1` باشد، یعنی سؤال است → `action_type_id = 1`
- اگر `posttypeid = 2` باشد، یعنی پاسخ است → `action_type_id = 2`

## ۲.۴.۴ مرحله ۳: استخراج رویدادهای PostHistory

جدول PostHistory کدهای خودش را دارد. ما باید آن‌ها را به کدهای یکسان خودمان تبدیل کنیم:

```python
PH_ACTION_MAP = {
    4: 13,   # Edit Title → 13
    5: 14,   # Edit Body → 14
    6: 15,   # Edit Tags → 15
    7: 16,   # Rollback Title → 16
    8: 17,   # Rollback Body → 17
    9: 18,   # Rollback Tags → 18
    10: 30,  # Close Post → 30
    11: 31,  # Reopen Post → 31
    12: 32,  # Delete → 32
    13: 33,  # Undelete → 33
    # ... و بقیه
}

# فقط کاربران فیلترشده‌ی ما را نگه دار
ph_user = (
    ph
    .join(users, left_on="userid", right_on="user_id", how="inner")
    .filter(pl.col("posthistorytypeid").is_in(list(PH_ACTION_MAP.keys())))
)

# تبدیل PostHistoryTypeId به ActionTypeId
events_ph = ph_user.with_columns([
    pl.col("posthistorytypeid")
    .replace(PH_ACTION_MAP)
    .cast(pl.Int32)
    .alias("action_type_id"),
])
```

### ۲.۴.۵ مرحله ۴: استخراج رویدادهای Comment

```python
COMMENT_ACTION = 70

events_comments = (
    comments
    .join(users, left_on="userid", right_on="user_id", how="inner")
    .with_columns([
        pl.lit(COMMENT_ACTION).alias("action_type_id"),  # همه نظرات = 70
    ])
)
```

### ۲.۴.۶ مرحله ۵: ترکیب و مرتب‌سازی

```
python
ترکیب سه منبع #
eventlog_lazy = pl.concat(
[events_create_post, events_ph, events_comments],
how="diagonal_relaxed"
)

مرتب‌سازی بر اساس کاربر و زمان #
eventlog_lazy = eventlog_lazy.sort(["user_id", "timestamp"])
```

**نتیجه:** یک جدول واحد که همه‌ی فعالیت‌های همه‌ی کاربران، به ترتیب زمانی، در آن است.

### ۲.۴.۷ مرحله ۶: محاسبه‌ی Gap Time

```
python
فاصله‌ی زمانی بین رویدادهای متوالی هر کاربر #
eventlog_lazy = eventlog_lazy.with_columns([
(pl.col("timestamp")
.diff()
.over("user_id") / (1000000*60*60*24))
.alias("gap_time_actions_days"),
(pl.col("timestamp")
.diff()
.over("user_id") / 1000000)
.alias("gap_time_actions_seconds"),
])
```

**معیار Gap Time چیست؟** فاصله‌ی زمانی بین دو فعالیت متوالی یک کاربر. این شاخص بسیار مهمی برای تشخیص Churn است.

> 💡 **مثال مفهومی:**
>
> کاربری که معمولاً هر ۳ روز یک فعالیت داشته، اگر ۳۰ روز غایب شود، gap_time_days از ۳ به ۳۰ می‌رسد — یک جهش ۱۰ برابری! این می‌تواند نشانه‌ی اولیه‌ی Churn باشد.

### ۲.۴.۸ مرحله ۷: ذخیره

```
python
Lazy اجرای واقعی همه‌ی عملیات‌های #
eventlog = eventlog_lazy.collect(engine="streaming")

Parquet ذخیره به فرمت #
eventlog.write_parquet("eventlog_v2.parquet")
```

---

## ۲.۵ ساختار نهایی Event Log

### ۲.۵.۱ ستون‌های Event Log

| ستون | نوع | توضیح ساده |
|---|---|---|
| `id` | INT | شماره ردیف (شناسه‌ی یکتای رویداد) |
| `user_id` | INT | شناسه‌ی کاربر |
| `timestamp` | DATETIME | تاریخ و ساعت دقیق رویداد |
| `action_type_id` | INT | نوع فعالیت (۱= Question, ۲=Answer, ۷۰=Comment, ...) |
| `post_id` | INT | شناسه‌ی پست مرتبط |
| `post_type_id` | INT | نوع پست (۱=Question, ۲=Answer) |
| `persona` | INT | پرسونای کاربر (۰ تا ۱۹ از NMF) |
| `LastAccessDate` | DATETIME | آخرین بازدید کاربر از سایت |
| `AccountCreationDate` | DATE | تاریخ عضویت |
| `gap_time_actions_days` | FLOAT | فاصله از رویداد قبلی (روز) |
| `gap_time_actions_seconds` | FLOAT | فاصله از رویداد قبلی (ثانیه) |

### ۲.۵.۲ نمونه‌ی رکورد

```
  id  | user_id | timestamp           | action_type_id | post_id | persona |
                                                                  gap_days
------|---------|---------------------|----------------|---------|---------|--
                                                                  ------
  1   | 12345   | 2020-06-01 08:30:00 | 2 (Answer)     | 98765   | 5       |
                                                                  null
  2   | 12345   | 2020-06-01 09:15:00 | 70 (Comment)   | 98760   | 5       |
                                                                  0.03
  3   | 12345   | 2020-06-03 14:20:00 | 14 (Edit_Body) | 98765   | 5       |
                                                                  2.21
  4   | 12345   | 2020-06-15 11:00:00 | 1 (Question)   | 99001   | 5       |
                                                                  11.86
```

**تفسیر:** کاربر ۱۲۳۴۵ در ۱ ژوئن پاسخ نوشت، ۴۵ دقیقه بعد نظر داد، ۲ روز بعد ویرایش کرد، و ۱۲ روز بعد سؤال پرسید.

---

# ۲.۶ ساخت Feature Matrix (فایل create_feature_matrix.py)

## ۲.۶.۱ هدف

از Event Log، ویژگی‌های **سطح-کاربر** استخراج می‌کنیم. یعنی برای هر کاربر، یک سطر با چندین ستون ویژگی.

## ۲.۶.۲ تعداد هر نوع فعالیت

```
python
کاربر per تعداد هر نوع فعالیت #
feature_action_counts = (
elog
.group_by(["user_id", "action_type_id"])
.agg(pl.len().alias("freq"))
)
```

**نتیجه:** ستون‌هایی مثل:

| ستون | توضیح |
|---|---|
| `count_action_1` | تعداد سؤالات این کاربر |
| `count_action_2` | تعداد پاسخ‌های این کاربر |
| `count_action_70` | تعداد نظرات این کاربر |
| `count_action_14` | تعداد ویرایش‌های بدنه |

| ستون | توضیح |
|---|---|
| ... | ... |

### ۲.۶.۳ آمار Gap Time

```
                                                                    python
                                                                ) = gap_stats
                                                                     elog
                                                            group_by("user_id").
                                                                   ])agg.
        ,pl.col("gap_time_actions_seconds").mean().alias("gap_mean")
          ,pl.col("gap_time_actions_seconds").std().alias("gap_std")
          ,pl.col("gap_time_actions_seconds").max().alias("gap_max")
          ,pl.col("gap_time_actions_seconds").min().alias("gap_min")
,pl.col("gap_time_actions_seconds").quantile(0.95).alias("gap_p95")
        ,pl.col("gap_time_actions_seconds").last().alias("gap_last")
                                                                  ([
                                                                      (
```

**توضیح ستون‌ها:**

| ستون | معنی |
|---|---|
| `gap_mean` | میانگین فاصله‌ی بین فعالیت‌ها |
| `gap_std` | انحراف معیار — آیا منظم است یا نامنظم؟ |
| `gap_max` | بزرگ‌ترین فاصله — طولانی‌ترین غیبت |
| `gap_last` | آخرین فاصله — الان چقدر است؟ |

---

## ۲.۷ خلاصه‌ی فصل

### ۲.۷.۱ فایل‌های تولیدشده

| فایل | توضیح | استفاده در |
|---|---|---|
| `eventlog_v2.parquet` | Event Log نهایی | همه‌ی فصل‌ها |
| `feature_matrix_v2.parquet` | ویژگی‌های سطح-کاربر | فصل ۳، ۴ |

### ۲.۷.۲ نکات کلیدی

1. ✅ **Vote** **حذف شد:** چون رأی‌دهنده مشخص نیست، فقط رأی‌گیرنده

2. ✅ **Initial_Title/Body/Tags حذف شد:** چون با Create_Question تکراری بود
3. ✅ **۲۶ نوع فعالیت** در Event Log ثبت می‌شود
4. ✅ **Gap Time** (فاصله از رویداد قبلی) برای هر رویداد محاسبه شده

### ۲.۷.۳ خروجی برای فصل بعد

Event Log برای محاسبه‌ی **Weighted Activity (WA)** در فصل ۳ و ساخت سیستم Rule-based استفاده می‌شود.